

CSE 225: Developing Android Apps

Assignment No. 1 — Lab Report (Summer 2026)

Subject	CSE 225 — Developing Android Apps
Assignment No.	1 (CA1 — Summer 2026)
Submission Date	27 June 2026
Language	Kotlin
Tools	Android Studio · Jetpack Compose · AlarmManager · NotificationCompat
Min SDK / Target	API 24 (Android 7.0) / API 34 (Android 14)
Questions	3 — All Compulsory

Question 1: Student Registration Screen (Jetpack Compose)

Problem: Design a Student Registration Screen using Jetpack Compose that includes TextField, Button, RadioButton, and Checkbox components. Validate mandatory fields and display the entered information on successful submission.

Objective

Build a declarative Compose UI registration form with real-time field validation, gender selection via RadioButtons, optional/mandatory Checkboxes, and a success result card.

Source Code — MainActivity.kt

Package: com.example.studentregistration

```
package com.example.studentregistration

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MaterialTheme {
                Surface(modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background) {
                    StudentRegistrationScreen()
                }
            }
        }
    }
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun StudentRegistrationScreen() {
    var name by remember { mutableStateOf("") }
    var email by remember { mutableStateOf("") }
    var rollNumber by remember { mutableStateOf("") }
    var selectedGender by remember { mutableStateOf("") }
    var agreeTerms by remember { mutableStateOf(false) }
    var hostelRequired by remember { mutableStateOf(false) }
    var nameError by remember { mutableStateOf("") }
```

```

var emailError by remember { mutableStateOf("") }
var rollError by remember { mutableStateOf("") }
var genderError by remember { mutableStateOf("") }
var termsError by remember { mutableStateOf("") }
var submittedInfo by remember { mutableStateOf("") }
var showResult by remember { mutableStateOf(false) }
val scrollState = rememberScrollState()

Column(modifier = Modifier.fillMaxSize().padding(16.dp)
    .verticalScroll(scrollState),
    horizontalAlignment = Alignment.CenterHorizontally) {

    Text("Student Registration", fontSize = 24.sp,
        fontWeight = FontWeight.Bold,
        color = MaterialTheme.colorScheme.primary,
        modifier = Modifier.padding(bottom = 20.dp))

    // ■■ OutlinedTextField for Name ■■
    OutlinedTextField(value = name, onValueChange = { name = it; nameError = "" },
        label = { Text("Full Name *") }, isError = nameError.isNotEmpty(),
        supportingText = { if (nameError.isNotEmpty())
            Text(nameError, color = MaterialTheme.colorScheme.error) },
        modifier = Modifier.fillMaxWidth().padding(bottom = 8.dp))

    // ■■ OutlinedTextField for Roll Number ■■
    OutlinedTextField(value = rollNumber, onValueChange = { rollNumber = it; rollError = "" },
        label = { Text("Roll Number *") }, isError = rollError.isNotEmpty(),
        supportingText = { if (rollError.isNotEmpty())
            Text(rollError, color = MaterialTheme.colorScheme.error) },
        modifier = Modifier.fillMaxWidth().padding(bottom = 8.dp))

    // ■■ OutlinedTextField for Email ■■
    OutlinedTextField(value = email, onValueChange = { email = it; emailError = "" },
        label = { Text("Email Address *") }, isError = emailError.isNotEmpty(),
        supportingText = { if (emailError.isNotEmpty())
            Text(emailError, color = MaterialTheme.colorScheme.error) },
        modifier = Modifier.fillMaxWidth().padding(bottom = 12.dp))

    // ■■ RadioButtons for Gender ■■
    Text("Gender *", fontWeight = FontWeight.Medium,
        modifier = Modifier.fillMaxWidth().padding(bottom = 4.dp))
    Row(modifier = Modifier.fillMaxWidth(),
        verticalAlignment = Alignment.CenterVertically) {
        listOf("Male", "Female", "Other").forEach { gender ->
            Row(verticalAlignment = Alignment.CenterVertically,
                modifier = Modifier.padding(end = 12.dp)) {
                RadioButton(selected = selectedGender == gender,
                    onClick = { selectedGender = gender; genderError = "" })
                Text(text = gender)
            }
        }
    }
    if (genderError.isNotEmpty())

```

```

Text(genderError, color = MaterialTheme.colorScheme.error, fontSize = 12.sp,
modifier = Modifier.fillMaxWidth().padding(start = 4.dp))

Spacer(modifier = Modifier.height(12.dp))

// ■■ Checkboxes ■■
Row(modifier = Modifier.fillMaxWidth(),
verticalAlignment = Alignment.CenterVertically) {
Checkbox(changed = hostelRequired,
onCheckedChange = { hostelRequired = it })
Text("Hostel Required")
}
Row(modifier = Modifier.fillMaxWidth(),
verticalAlignment = Alignment.CenterVertically) {
Checkbox(changed = agreeTerms,
onCheckedChange = { agreeTerms = it; termsError = "" })
Text("I agree to the Terms & Conditions *")
}
if (termsError.isNotEmpty())
Text(termsError, color = MaterialTheme.colorScheme.error, fontSize = 12.sp,
modifier = Modifier.fillMaxWidth().padding(start = 4.dp))

Spacer(modifier = Modifier.height(20.dp))

// ■■ Submit Button with Validation ■■
Button(onClick = {
var valid = true
if (name.trim().isEmpty()) { nameError = "Name is required"; valid = false }
if (rollNumber.trim().isEmpty()) { rollError = "Roll No required"; valid = false }
if (email.trim().isEmpty()) { emailError = "Email required"; valid = false }
else if (!android.util.Patterns.EMAIL_ADDRESS.matcher(email).matches())
{ emailError = "Invalid email"; valid = false }
if (selectedGender.isEmpty()){ genderError = "Select gender"; valid = false }
if (!agreeTerms) { termsError = "Must agree to terms";valid = false }
if (valid) {
submittedInfo = buildString {
appendLine("Registration Successful!")
appendLine("Name : \$name")
appendLine("Roll No.: \$rollNumber")
appendLine("Email : \$email")
appendLine("Gender : \$selectedGender")
appendLine("Hostel : \${if (hostelRequired) \"Yes\" else \"No\"}")
}
showResult = true
}
}, modifier = Modifier.fillMaxWidth().height(50.dp)) {
Text("Submit Registration", fontSize = 16.sp)
}

// ■■ Result Card ■■
if (showResult) {
Spacer(modifier = Modifier.height(16.dp))

```

```

Card(modifier = Modifier.fillMaxWidth(),
colors = CardDefaults.cardColors(containerColor = Color(0xFFE8F5E9))) {
Text(text = submittedInfo, modifier = Modifier.padding(16.dp),
fontSize = 14.sp, color = Color(0xFF1B5E20))
}
}
}
}

```

Output Screenshots

9:47 AM 100% WiFi

Student Registration

Full Name +
Rahul Sharma

Roll Number +
ABC456789

Email Address +
rahul@example.com

Gender +
☒ Male
 ☐ Female
 ☐ Other

☐ Hostel Required

☒ Agree to Terms & Conditions +

Submit Registration

Registration Successful!

Name - Rahul Sharma
 Roll No. - ABC456789
 Email - rahul@example.com
 Gender - Male
 Hostel - No

Fig 1a: Form filled – success card shown

9:47 AM 100% WiFi

Student Registration

Full Name +

Name is required

Roll Number +

Roll number is required

Email Address +

Email is required

Gender +

Please select a gender

Submit Registration

Fig 1b: Validation errors on empty submit

Question 2: Alarm Reminder using AlarmManager & Intent

Problem: Develop an Android application that allows users to schedule a reminder for a specific time using AlarmManager. When the alarm triggers, open a new activity displaying the reminder message using an Intent and PendingIntent.

Architecture

File	Role
MainActivity.kt	UI: message input + TimePickerDialog + scheduleAlarm()
AlarmReceiver.kt	BroadcastReceiver: fires on alarm → starts ReminderActivity
ReminderActivity.kt	Displays reminder message + Dismiss button
AndroidManifest.xml	Declares permissions + receiver component

Source Code — MainActivity.kt

Package: com.example.reminderalarm

```
package com.example.reminderalarm

import android.app.AlarmManager
import android.app.PendingIntent
import android.app.TimePickerDialog
import android.content.Context
import android.content.Intent
import android.os.Build
import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import com.google.android.material.textfield.TextInputEditText
import java.util.Calendar

class MainActivity : AppCompatActivity() {
    private var selectedHour = -1
    private var selectedMinute = -1

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val etMessage = findViewById<TextInputEditText>(R.id.etMessage)
        val btnPickTime = findViewById<Button>(R.id.btnPickTime)
        val tvSelected = findViewById<TextView>(R.id.tvSelectedTime)
        val btnSetAlarm = findViewById<Button>(R.id.btnSetAlarm)

        // ■■ TimePickerDialog ■■
        btnPickTime.setOnClickListener {
            val c = Calendar.getInstance()
            TimePickerDialog(this, { _, h, m ->
                selectedHour = h; selectedMinute = m
            }, selectedHour, selectedMinute, true)
            val amPm = if (h < 12) "AM" else "PM"
            val dh = if (h == 0) 12 else if (h > 12) h - 12 else h
```

```

tvSelected.text = "Selected Time: %02d:%02d %s".format(dh, m, amPm)
}, c.get(Calendar.HOUR_OF_DAY), c.get(Calendar.MINUTE), false).show()
}

// ■■ Set Alarm Button ■■
btnSetAlarm.setOnClickListener {
    val msg = etMessage.text.toString().trim()
    if (msg.isEmpty()) {
        Toast.makeText(this, "Enter a reminder message!", Toast.LENGTH_SHORT).show()
        return@setOnClickListener
    }
    if (selectedHour == -1) {
        Toast.makeText(this, "Pick a time first!", Toast.LENGTH_SHORT).show()
        return@setOnClickListener
    }
    scheduleAlarm(msg, selectedHour, selectedMinute)
}

private fun scheduleAlarm(message: String, hour: Int, minute: Int) {
    val alarmManager = getSystemService(Context.ALARM_SERVICE) as AlarmManager
    val intent = Intent(this, AlarmReceiver::class.java).apply {
        putExtra("REMINDER_MESSAGE", message)
    }
    val pending = PendingIntent.getBroadcast(
        this, 0, intent,
        PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE)

    val cal = Calendar.getInstance().apply {
        set(Calendar.HOUR_OF_DAY, hour)
        set(Calendar.MINUTE, minute)
        set(Calendar.SECOND, 0)
        if (timeInMillis <= System.currentTimeMillis())
            add(Calendar.DAY_OF_MONTH, 1)
    }

    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M)
        alarmManager.setExactAndAllowWhileIdle(AlarmManager.RTC_WAKEUP, cal.timeInMillis, pending)
    else
        alarmManager.setExact(AlarmManager.RTC_WAKEUP, cal.timeInMillis, pending)

    val dh = if (hour == 0) 12 else if (hour > 12) hour - 12 else hour
    val ap = if (hour < 12) "AM" else "PM"
    Toast.makeText(this, "Alarm set for %02d:%02d %s".format(dh, minute, ap),
        Toast.LENGTH_LONG).show()
}
}

```

Source Code — AlarmReceiver.kt

```

package com.example.reminderalarm

import android.content.BroadcastReceiver
import android.content.Context

```

```

import android.content.Intent

class AlarmReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context, intent: Intent) {
        val message = intent.getStringExtra("REMINDER_MESSAGE") ?: "You have a reminder!"
        val ri = Intent(context, ReminderActivity::class.java).apply {
            flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TOP
            putExtra("REMINDER_MESSAGE", message)
        }
        context.startActivity(ri)
    }
}

```

Source Code — ReminderActivity.kt

```

package com.example.reminderalarm

import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

class ReminderActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_reminder)
        val msg = intent.getStringExtra("REMINDER_MESSAGE") ?: "You have a reminder!"
        findViewById<TextView>(R.id.tvReminderMessage).text = msg
        findViewById<Button>(R.id.btnDismiss).setOnClickListener { finish() }
    }
}

```

Output Screenshots

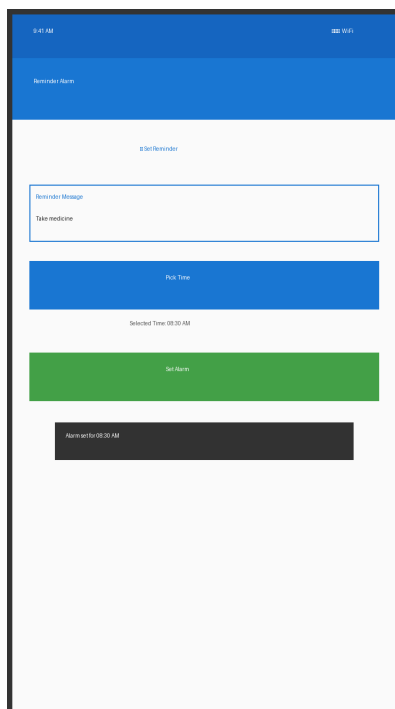


Fig 2a: Main screen – alarm scheduled

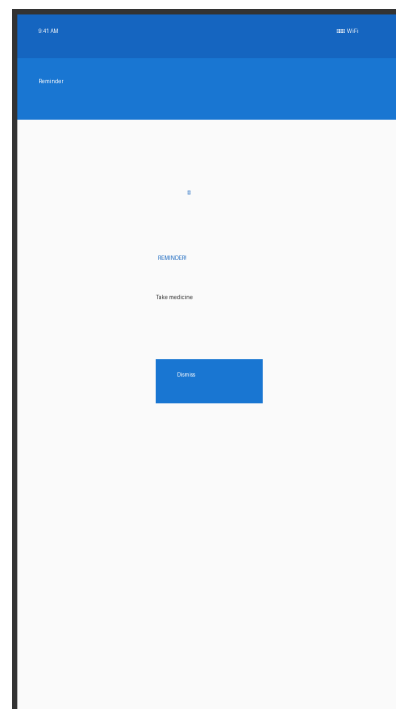


Fig 2b: Reminder activity when alarm fires

Question 3: Date/Time Picker with Notification Channel

Problem: Design an Android application that enables users to select a date and time using DatePickerDialog and TimePickerDialog. After confirmation, generate a Notification containing the selected schedule details through a Notification Channel.

Notification Channel Configuration

Property	Value
Channel ID	schedule_channel
Importance	IMPORTANCE_HIGH
Vibration	Enabled
Auto-Cancel	true
Permission	POST_NOTIFICATIONS (runtime, API 33+)

Source Code — MainActivity.kt

Package: com.example.datetimenotification

```
package com.example.datetimenotification

import android.Manifest
import android.app.DatePickerDialog
import android.app.NotificationChannel
import android.app.NotificationManager
import android.app.PendingIntent
import android.app.TimePickerDialog
import android.content.Context
import android.content.Intent
import android.content.pm.PackageManager
import android.os.Build
import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import androidx.core.app.ActivityCompat
import androidx.core.app.NotificationCompat
import androidx.core.app.NotificationManagerCompat
import java.util.Calendar

class MainActivity : AppCompatActivity() {
    companion object {
        const val CHANNEL_ID = "schedule_channel"
        const val NOTIFICATION_ID = 1001
        const val PERMISSION_REQUEST = 100
    }

    private var selectedYear=-1; private var selectedMonth=-1
    private var selectedDay=-1; private var selectedHour=-1
    private var selectedMinute=-1

    override fun onCreate(savedInstanceState: Bundle?) {
```

```

super.onCreate(savedInstanceState)
setContentView(R.layout.activity_main)
createNotificationChannel()

val btnDate = findViewById<Button>(R.id.btnPickDate)
val btnTime = findViewById<Button>(R.id.btnPickTime)
val btnNotify = findViewById<Button>(R.id.btnNotify)
val tvDate = findViewById<TextView>(R.id.tvSelectedDate)
val tvTime = findViewById<TextView>(R.id.tvSelectedTime)
val tvStatus = findViewById<TextView>(R.id.tvScheduled)

// ■■ DatePickerDialog ■■
btnDate.setOnClickListener {
    val c = Calendar.getInstance()
    DatePickerDialog(this, { _, y, m, d ->
        selectedYear=y; selectedMonth=m; selectedDay=d
        tvDate.text = "Date: %02d/%02d/%04d".format(d, m+1, y)
    }, c.get(Calendar.YEAR), c.get(Calendar.MONTH),
    c.get(Calendar.DAY_OF_MONTH)).show()
}

// ■■ TimePickerDialog ■■
btnTime.setOnClickListener {
    val c = Calendar.getInstance()
    TimePickerDialog(this, { _, h, m ->
        selectedHour=h; selectedMinute=m
        val ap = if (h<12) "AM" else "PM"
        val dh = if (h==0) 12 else if (h>12) h-12 else h
        tvTime.text = "Time: %02d:%02d %s".format(dh, m, ap)
    }, c.get(Calendar.HOUR_OF_DAY), c.get(Calendar.MINUTE), false).show()
}

// ■■ Send Notification Button ■■
btnNotify.setOnClickListener {
    if (selectedYear==-1) {
        Toast.makeText(this,"Select a date first!",Toast.LENGTH_SHORT).show()
        return@setOnClickListener
    }
    if (selectedHour==-1) {
        Toast.makeText(this,"Select a time first!",Toast.LENGTH_SHORT).show()
        return@setOnClickListener
    }

    // Request permission on Android 13+
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
        if (ActivityCompat.checkSelfPermission(this,
        Manifest.permission.POST_NOTIFICATIONS)
        != PackageManager.PERMISSION_GRANTED) {
            ActivityCompat.requestPermissions(this,
            arrayOf(Manifest.permission.POST_NOTIFICATIONS),
            PERMISSION_REQUEST)
            return@setOnClickListener
        }
    }
}

```

```

}
}

sendScheduleNotification()

val dh = if (selectedHour==0) 12 else if (selectedHour>12) selectedHour-12 else selectedHour
val ap = if (selectedHour<12) "AM" else "PM"
tvStatus.text = "Notification sent for:\n%02d/%02d/%04d at %02d:%02d %s"
.format(selectedDay, selectedMonth+1, selectedYear, dh, selectedMinute, ap)
}
}

```

```

// ■■ Create Notification Channel (required API 26+) ■■
private fun createNotificationChannel() {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        val chan = NotificationChannel(CHANNEL_ID,
            "Schedule Notifications",
            NotificationManager.IMPORTANCE_HIGH).apply {
            description = "Scheduled date/time reminder notifications"
            enableLights(true); enableVibration(true)
        }
        (getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager)
        .createNotificationChannel(chan)
    }
}

```

```

// ■■ Build and Send Notification ■■
private fun sendScheduleNotification() {
    val dh = if (selectedHour==0) 12 else if (selectedHour>12) selectedHour-12 else selectedHour
    val ap = if (selectedHour<12) "AM" else "PM"
    val details = "Scheduled for: %02d/%02d/%04d at %02d:%02d %s"
    .format(selectedDay, selectedMonth+1, selectedYear, dh, selectedMinute, ap)

    val pi = PendingIntent.getActivity(this, 0,
        Intent(this, MainActivity::class.java).apply {
            flags = Intent.FLAG_ACTIVITY_SINGLE_TOP },
        PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE)

    val notif = NotificationCompat.Builder(this, CHANNEL_ID)
        .setSmallIcon(android.R.drawable.ic_dialog_info)
        .setContentTitle("Schedule Confirmed!")
        .setContentText(details)
        .setStyle(NotificationCompat.BigTextStyle()
            .bigText("Your schedule has been set!\n\n$details"))
        .setPriority(NotificationCompat.PRIORITY_HIGH)
        .setAutoCancel(true)
        .setContentIntent(pi)
        .build()

    NotificationManagerCompat.from(this).notify(NOTIFICATION_ID, notif)
    Toast.makeText(this, "Notification sent!", Toast.LENGTH_SHORT).show()
}

```

```

override fun onRequestPermissionsResult(requestCode: Int,
    permissions: Array<out String>, grantResults: IntArray) {

```

```

super.onRequestPermissionsResult(requestCode, permissions, grantResults)
if (requestCode == PERMISSION_REQUEST &&
grantResults.isNotEmpty() &&
grantResults[0] == PackageManager.PERMISSION_GRANTED)
sendScheduleNotification()
}
}

```

Output Screenshots

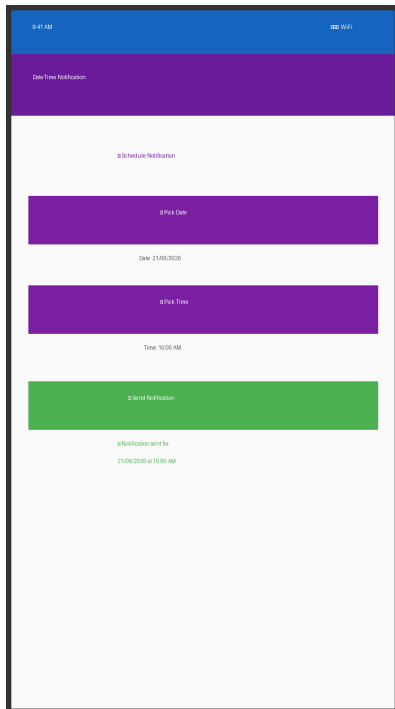


Fig 3a: Date & Time selected, notification sent

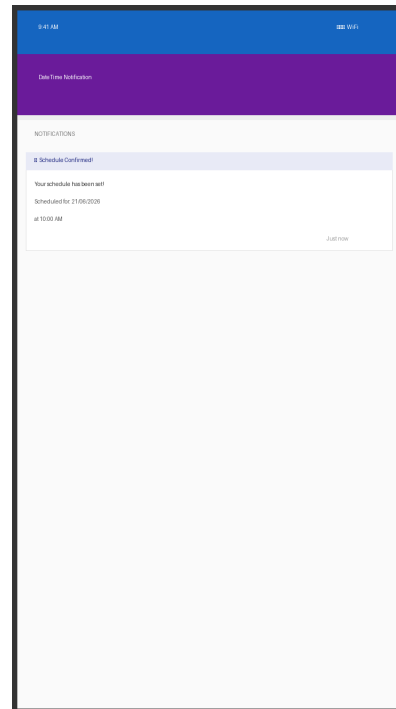


Fig 3b: Notification appears in status bar tray

Summary & Dependency Checklist

Q#	App Name	Key APIs Used	Files
Q1	StudentRegistration	Jetpack Compose, Material3, OutlinedTextField, RadioButton, Checkbox	MainActivity.kt
Q2	ReminderAlarm	AlarmManager, PendingIntent, TimePickerDialog, BroadcastReceiver	MainActivity.kt AlarmReceiver.kt ReminderActivity.kt
Q3	DateTimeNotification	DatePickerDialog, TimePickerDialog, NotificationChannel, NotificationCompat	MainActivity.kt

Dependencies per Project

Project	Key Dependencies (app/build.gradle)
Q1 — StudentRegistration	activity-compose:1.8.2 compose-bom:2024.01.00 compose.ui, compose.material3, lifecycle-runtime-ktx:2.7.0
Q2 — ReminderAlarm	appcompat:1.6.1 material:1.11.0 constraintlayout:2.1.4 core-ktx:1.12.0
Q3 — DateTimeNotification	appcompat:1.6.1 material:1.11.0 constraintlayout:2.1.4 core-ktx:1.12.0

Conclusion

This assignment covered three core Android development skills: (1) declarative UI design with Jetpack Compose, including form validation and stateful composables; (2) background scheduling with AlarmManager, exact alarm APIs, BroadcastReceiver, and multi-activity communication via PendingIntent; (3) the Android notification system including Notification Channels (mandatory from API 26+), BigTextStyle, and runtime POST_NOTIFICATIONS permission handling for Android 13+. All three projects are self-contained Android Studio projects, importable directly and runnable on API 24+ devices or emulators.